RaspberryPi2EGLFS Cross-Compile and Qt Setup Guide

Danial Mobini

2025-12-08

Table of Contents

RaspberryPi2EGLFS Cross-Compile and Qt Setup Guide

Host PC vs Raspberry Pi Steps

- **[on host PC]**: Run these commands on your Linux desktop or build machine.
- **[on RPi]**: Run these commands on your Raspberry Pi device.

1. Prepare Raspbian Image

[on RPi]

- Download old or latest Raspbian images.
- Unzip and write to SD card:

```
sudo dd if=2015-09-24-raspbian-jessie.img of=/dev/mmcblk0 bs=4M
```

- (Optional) Configure Pi:

```
sudo raspi-config
```

  - Boot to console
  - Set GPU memory to 256 MB

- For Raspbian Stretch, update firmware:

```
sudo rpi-update
reboot
```

2. Install Development Files on Pi

[on RPi]

- Edit `/etc/apt/sources.list` and uncomment `deb-src` line.
- Update and install dependencies:

```
sudo apt-get update
sudo apt-get build-dep qt4-x11
sudo apt-get build-dep libqt5gui5
sudo apt-get install libudev-dev libinput-dev libts-dev libxcb-xinerama
```

3. Prepare Target Directory on Pi

[on RPi]

```
sudo mkdir /usr/local/qt5pi
sudo chown pi:pi /usr/local/qt5pi
```

4. Prepare Host PC

[on host PC]

- Create working directory and get toolchain:

  ```
  mkdir ~/raspi
  cd ~/raspi
  git clone https://github.com/raspberrypi/tools
  ```

5. Create and Sync Sysroot

[on host PC]

```
mkdir /usr/local/rpi-sysroot /usr/local/rpi-sysroot/usr /usr/local/rpi-sysr
rsync -avz root@350g:/lib /usr/local/rpi-sysroot
rsync -avz root@350g:/usr/include /usr/local/rpi-sysroot/usr
rsync -avz root@350g:/usr/lib /usr/local/rpi-sysroot/usr
rsync -avz root@350g:/opt/vc /usr/local/rpi-sysroot/opt
```

6. Fix Sysroot Symlinks

[on host PC]

```
wget https://raw.githubusercontent.com/Kukkimonsuta/rpi-buildqt/master/scri
chmod +x sysroot-relativelinks.py
./sysroot-relativelinks.py /usr/local/rpi-sysroot
```

7. Get Qt and Configure

[on host PC]

Qt Shell Commands

```
git clone git://code.qt.io/qt/qtbase.git -b <qt-version>
cd qtbase
./configure -release -opengl es2 -device <rpi-version> \
-device-option CROSS_COMPILE=~/raspi/tools/arm-bcm2708/gcc-linaro-arm-linux
-sysroot ~/raspi/sysroot -opensource -confirm-license -make libs \
-prefix /usr/local/qt5pi -extprefix ~/raspi/qt5pi -hostprefix ~/raspi/qt5 -
make
make install
```

Shell Commands

```
git clone git://code.qt.io/qt/qtbase.git -b 5.12
cd qtbase

./configure -release -opengl es2 -device linux-rasp-pi3-g++ -device-option
```

add `#include <limits>` if you get errors about `std::numeric_limits` during configure.

```
make
make install
```

- Clean build if needed:

  ```
  git clean -dfx
  ```

8. Deploy Qt to Device

[on host PC]

```
cd ..
rsync -avz /usr/local/ext-qt5pi/ root@350g:/usr/local/qt5pi/
```

9. Build and Test Example

[on host PC]

```
cd qtbase/examples/opengl/qopenglwidget
~/raspi/qt5/bin/qmake
make
scp qopenglwidget root@350g:/home/pi
```

10. Update Linker on Pi

[on RPi]

```
echo /usr/local/qt5pi/lib | tee /etc/ld.so.conf.d/qt5pi.conf
ldconfig
```

11. Fix EGL/GLES Libraries

[on RPi]

```
mv /usr/lib/arm-linux-gnueabihf/libEGL.so.1.0.0 /usr/lib/arm-linux-gnueabih
mv /usr/lib/arm-linux-gnueabihf/libGLESv2.so.2.0.0 /usr/lib/arm-linux-gnuea
ln -s /opt/vc/lib/libEGL.so /usr/lib/arm-linux-gnueabihf/libEGL.so.1.0.0
ln -s /opt/vc/lib/libGLESv2.so /usr/lib/arm-linux-gnueabihf/libGLESv2.so.2.
```

```
ln -s /opt/vc/lib/libbrcmEGL.so /opt/vc/lib/libEGL.so
ln -s /opt/vc/lib/libbrcmGLESv2.so /opt/vc/lib/libGLESv2.so
ln -s /opt/vc/lib/libEGL.so /opt/vc/lib/libEGL.so.1
ln -s /opt/vc/lib/libGLESv2.so /opt/vc/lib/libGLESv2.so.2
```

12. Run Example on Pi

- Run the example, should work fullscreen with input support.

13. Build Other Qt Modules

```
git clone git://code.qt.io/qt/<qt-module>.git -b <qt-version>
cd <qt-module>
/usr/local/host-qt5/bin/qmake
make
make install
rsync -avz /usr/local/ext-qt5pi/ root@350g:/usr/local/qt5pi/
```

```
git clone git://code.qt.io/qt/qtdeclarative.git -b 5.12
git clone git://code.qt.io/qt/qtquickcontrols.git -b 5.12
git clone git://code.qt.io/qt/qtserialport.git -b 5.12
git clone git://code.qt.io/qt/qtquickcontrols2.git -b 5.12
git clone git://code.qt.io/qt/qtsvg.git -b 5.12
git clone git://code.qt.io/qt/qtvirtualkeyboard.git -b 5.12
```

```
cd qtdeclarative
/usr/local/host-qt5/bin/qmake
make
sudo make install

cd ../qtquickcontrols
/usr/local/host-qt5/bin/qmake
make
sudo make install
cd ../qtserialport
/usr/local/host-qt5/bin/qmake
make
sudo make install

cd ../qtquickcontrols2
/usr/local/host-qt5/bin/qmake
make
sudo make install

cd ../qtsvg
```

```
/usr/local/host-qt5/bin/qmake
make
sudo make install

cd ../qtvirtualkeyboard
/usr/local/host-qt5/bin/qmake
make
sudo make install

rsync -avz /usr/local/ext-qt5pi/ root@350g:/usr/local/qt5pi/
```

add `#include <limits>` if you get errors about `std::numeric_limits` during make.

Additional Notes

- For low resolution issues, add `disable_overscan=1` to `/boot/config.txt` and use `tvservice/fbset` to set resolution.
- Enable Qt logging for debugging:

  ```
  export QT_LOGGING_RULES=qt.qpa.*=true
  ./qopenglwidget
  ```

- Check `config.summary` for enabled features (Evdev, FontConfig, FreeType, libinput, EGLFS, EGLFS Raspberry Pi, LinuxFB, udev, xkbcommon-evdev).

Troubleshooting

- If EGLFS is missing, use `linux-rasp-pi-g++` mkspec and copy Pi 3 build flags.
- Fix VC paths in `qmake.conf` if needed.

Environment Variables

If Qt apps fail to find platform plugins:

```
export QT_QPA_PLATFORM=eglfs
export QT_QPA_PLATFORM_PLUGIN_PATH=/usr/local/qt5pi/plugins/platforms
export LD_LIBRARY_PATH=/usr/local/qt5pi/lib
```

Qt Creator Setup

- Add device, compiler, debugger, Qt version, and kit in Qt Creator.
- Use deploy steps and custom run configuration if needed.

Sense HAT

- Use unofficial Qt Sense HAT module for sensors and LEDs.

Qt Multimedia

- GStreamer-based multimedia may be unstable.
- Try OpenMAX for video/camera.
- Install GStreamer dependencies before building Qt Multimedia:

  ```
  sudo apt-get install gstreamer1.0-omx libgstreamer1.0-dev libgstreamer-
  ```

- Sync headers/libs to sysroot and re-run symlink script.

Example: Build Qt Multimedia with GStreamer 1.0

```
~/raspi/qt5/bin/qmake -r GST_VERSION=1.0
make
make install
```

- For OpenMAX debugging:

  ```
  export GST_DEBUG=omx:4
  ```